Volcano for the Agentic AI Era:

Unified Scheduling Across Training, Inference, and Agents

Ricardo Aravena · Rainie Li

KubeCon + CloudNativeCon North America 2026
Salt Lake City · November 9–12

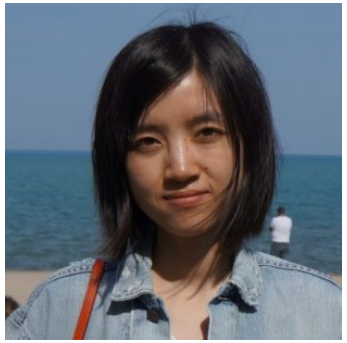
Speakers



Ricardo Aravena

AI Infrastructure Lead · Nunchux AI

CNCF TOC Alternate (GB)



Rainie Li

Pinterest

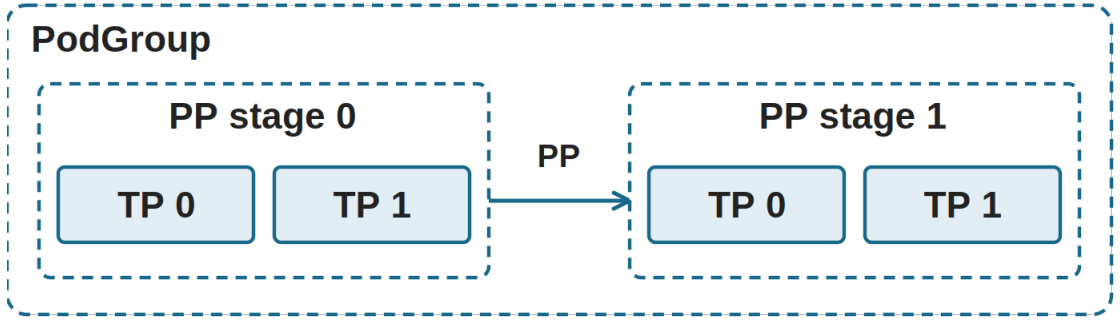
Apache YuniKorn PMC member

01

Unified scheduling

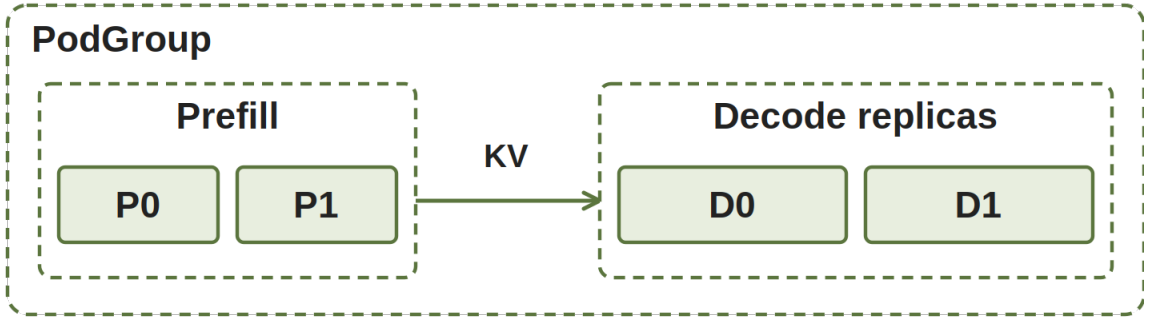
Different workloads, shared infrastructure

Training



Place complete TP / PP groups.

Inference



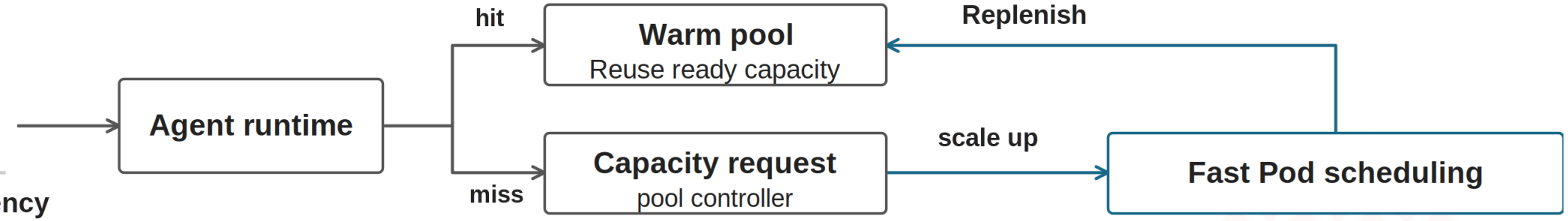
Keep P/D communication local.

Agents

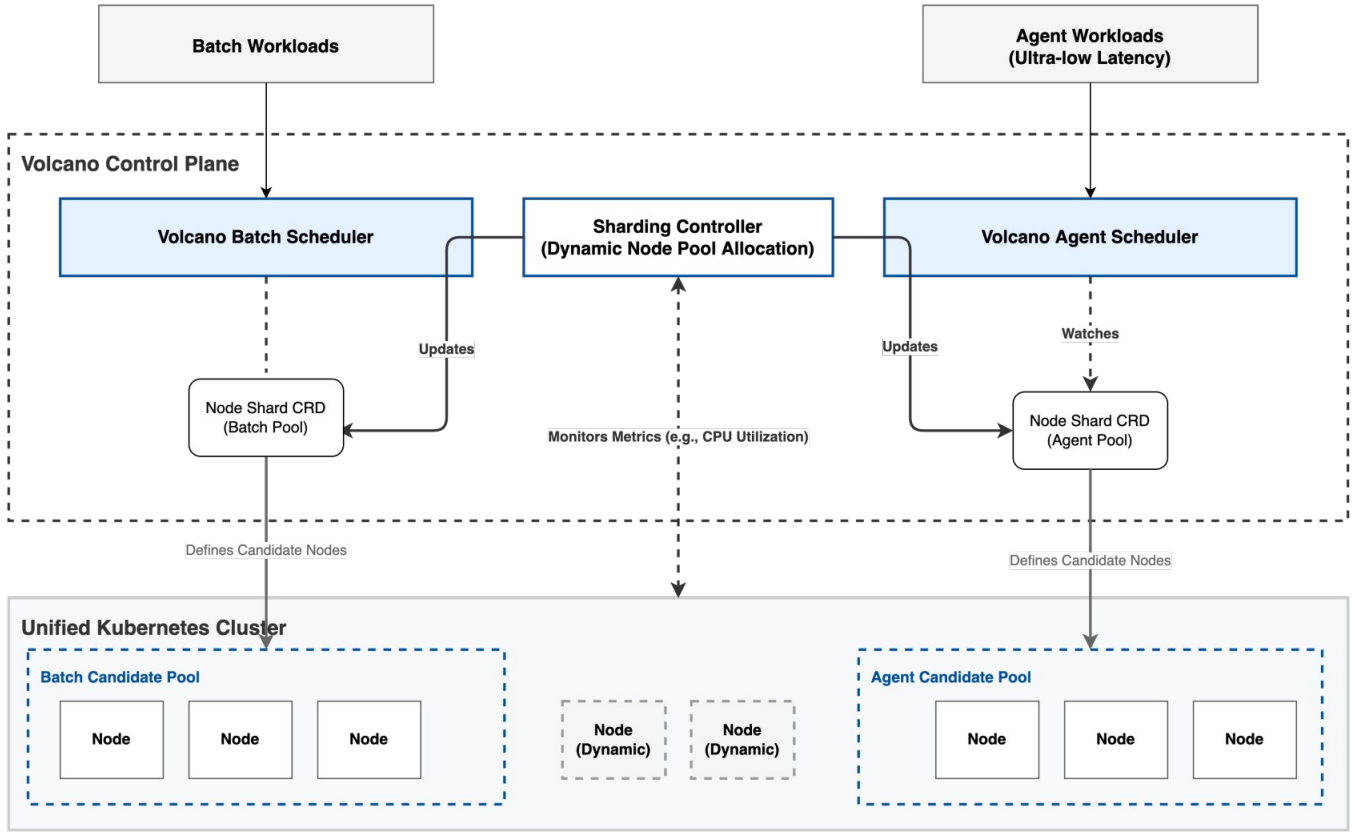
Bursty arrivals



Low startup latency



Unified scheduling



Batch and fast paths

Batch jobs and latency-sensitive Pods use separate paths.

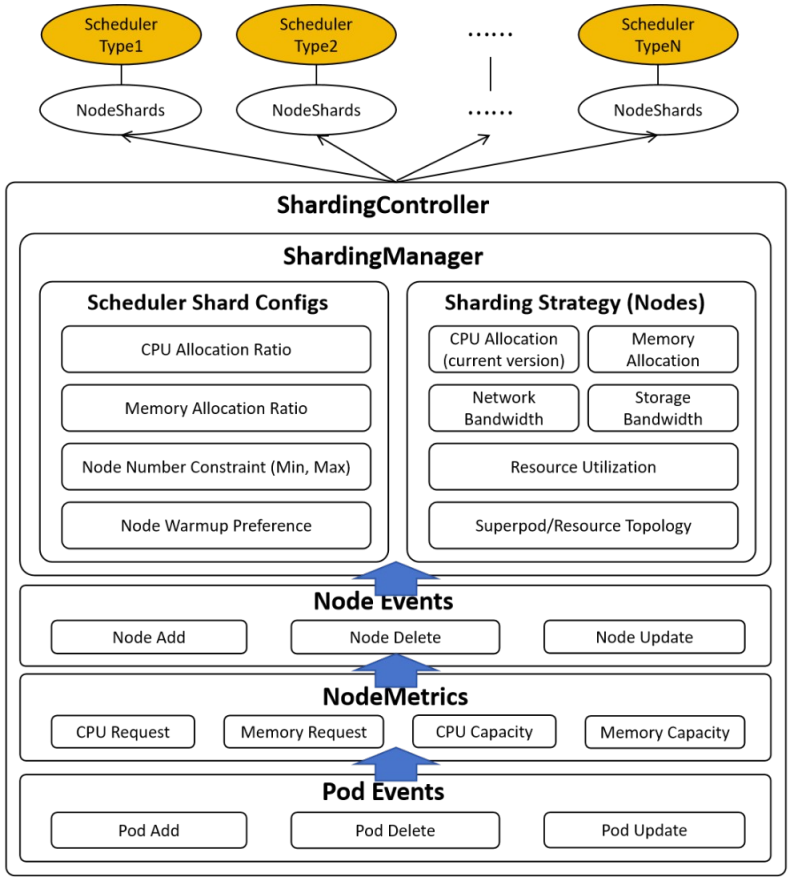
Binder conflict checks

Reject conflicting placements before binding.

NodeShard coordination

Hand off nodes without migrating running Pods.

NodeShard coordination



NodeShard

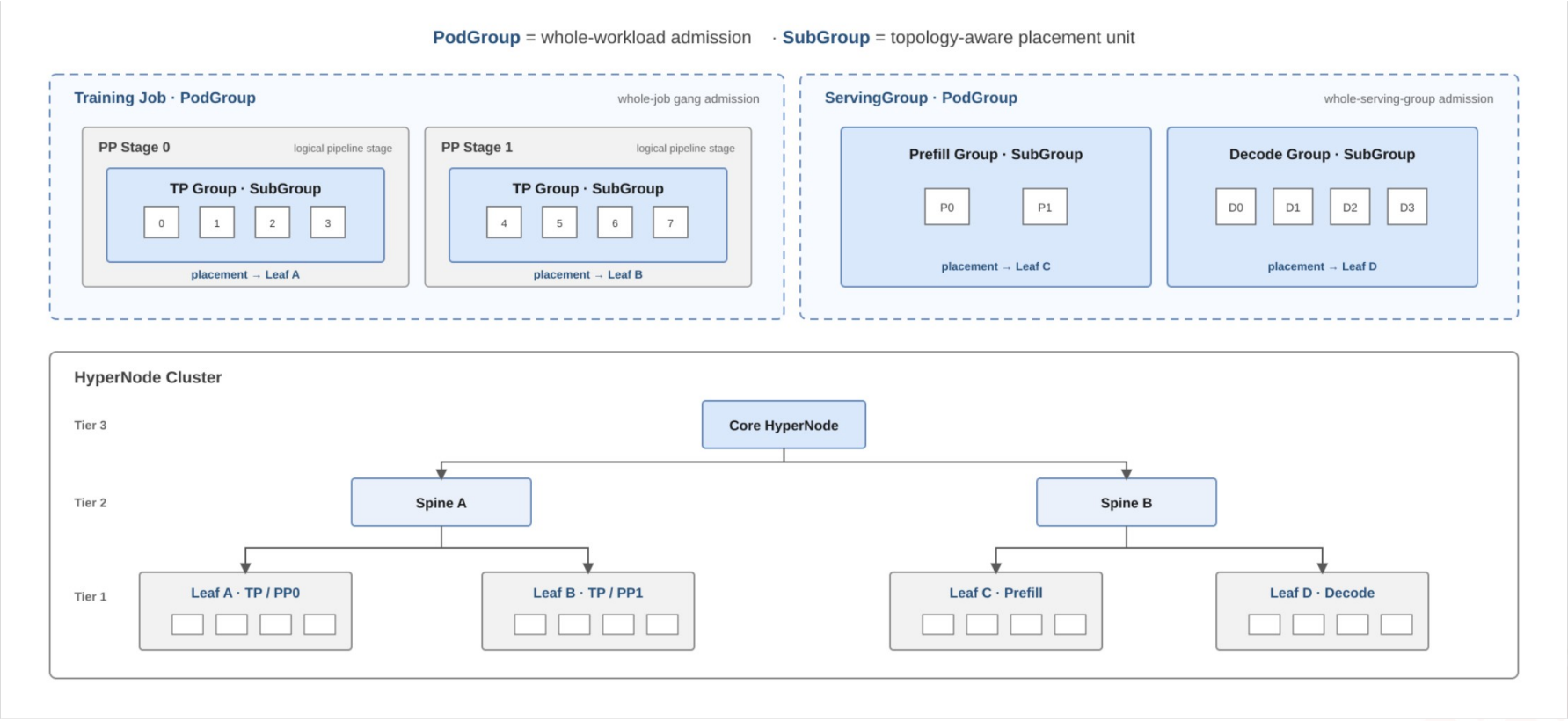
```
spec:
  nodesDesired:
  - node1
  - node2
status:
  nodesInUse:
  - node0
  - node1
  nodesToRemove:
  - node0
  nodesToAdd:
  - node2
```

Desired nodes become usable after handoff.

02

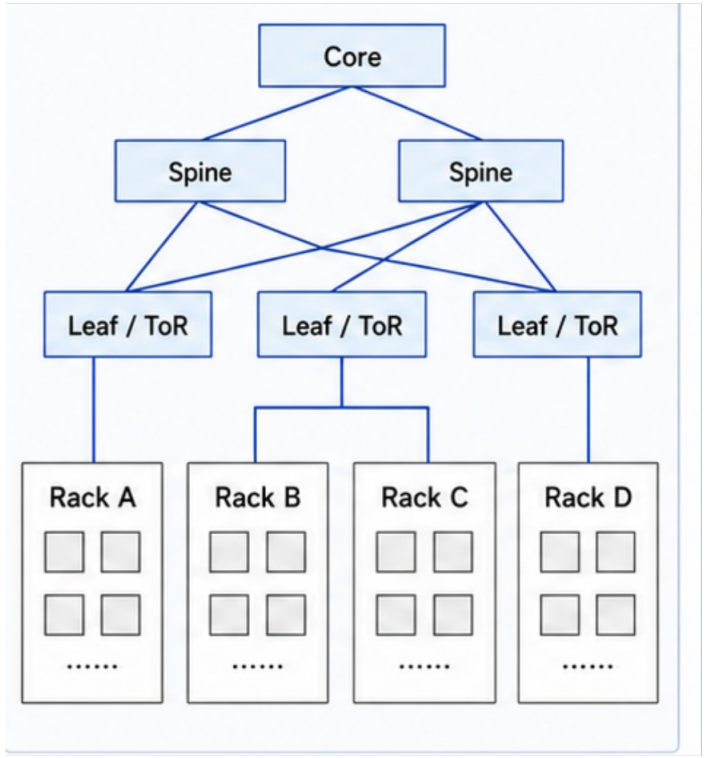
Multi-level gang & network topology

PodGroup and SubGroup



PodGroup sets the workload minimum. SubGroups keep each replica complete.

HyperNode



Parent HyperNode

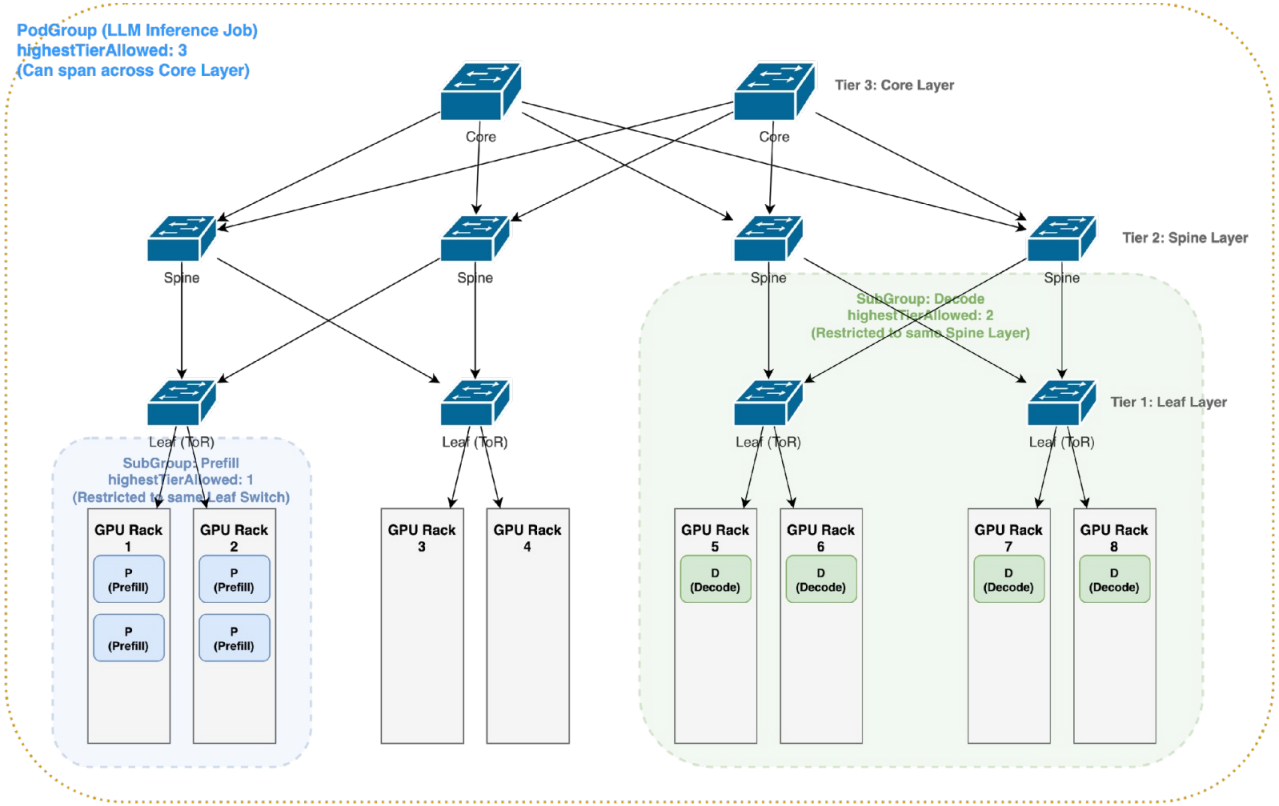
```
tier: 2
tierName: spine
members:
- type: HyperNode
  selector:
    exactMatch:
      name: rack-a
```

Leaf HyperNode

```
tier: 1
tierName: leaf
members:
- type: Node
  selector:
    labelMatch:
      matchLabels:
        rack: rack-a
```

A parent groups HyperNodes. A leaf groups Nodes.

Place groups at different topology levels

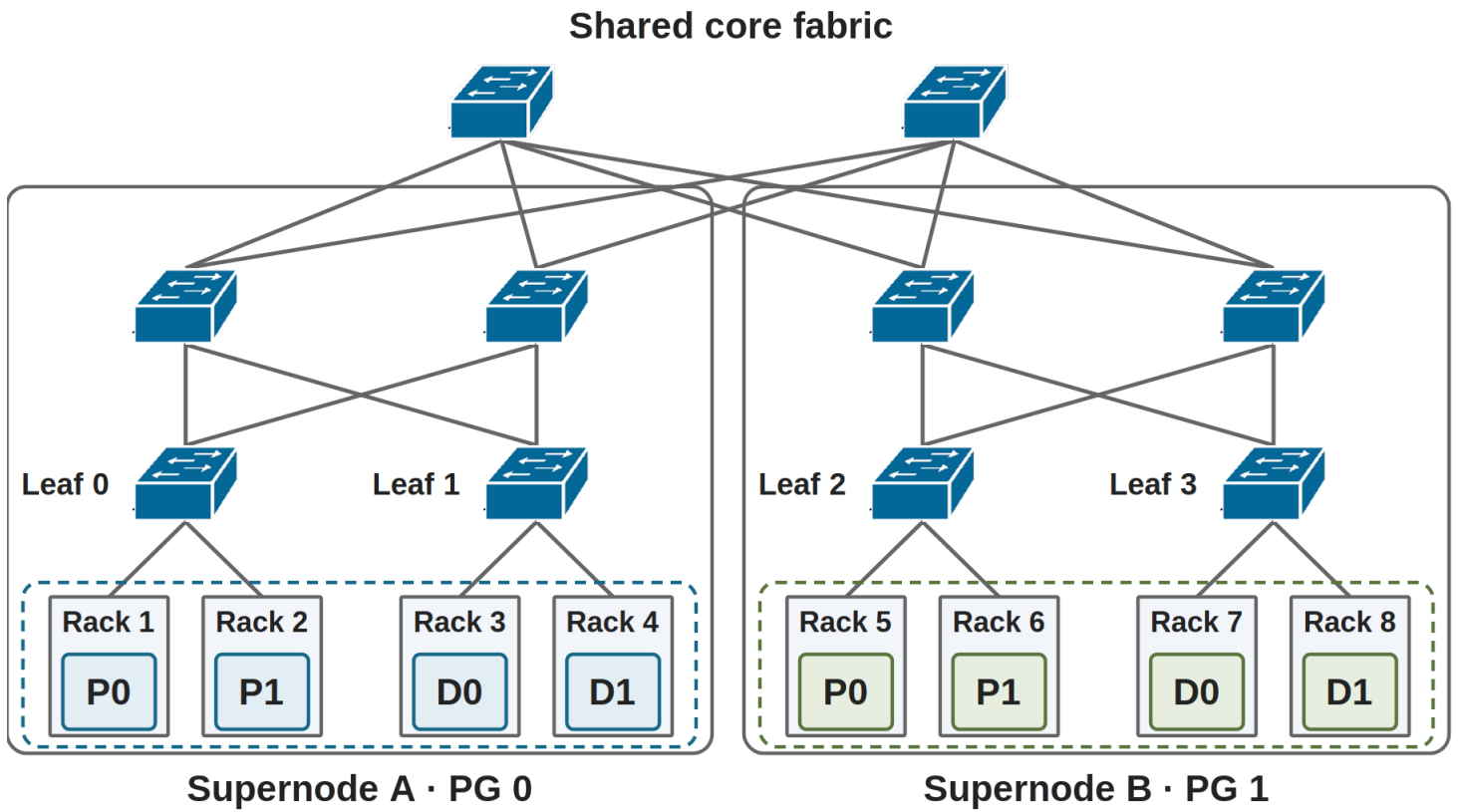


Prefill stays under one leaf. Decode can span leaves.

PodGroup.spec

```
minMember: 8
networkTopology:
  mode: hard
  highestTierName: core
subGroupPolicy:
- name: prefill
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: leaf
- name: decode
  subGroupSize: 4
  networkTopology:
    mode: hard
    highestTierName: spine
```

PodGroup anti-affinity



A supernode failure leaves the other instance available.

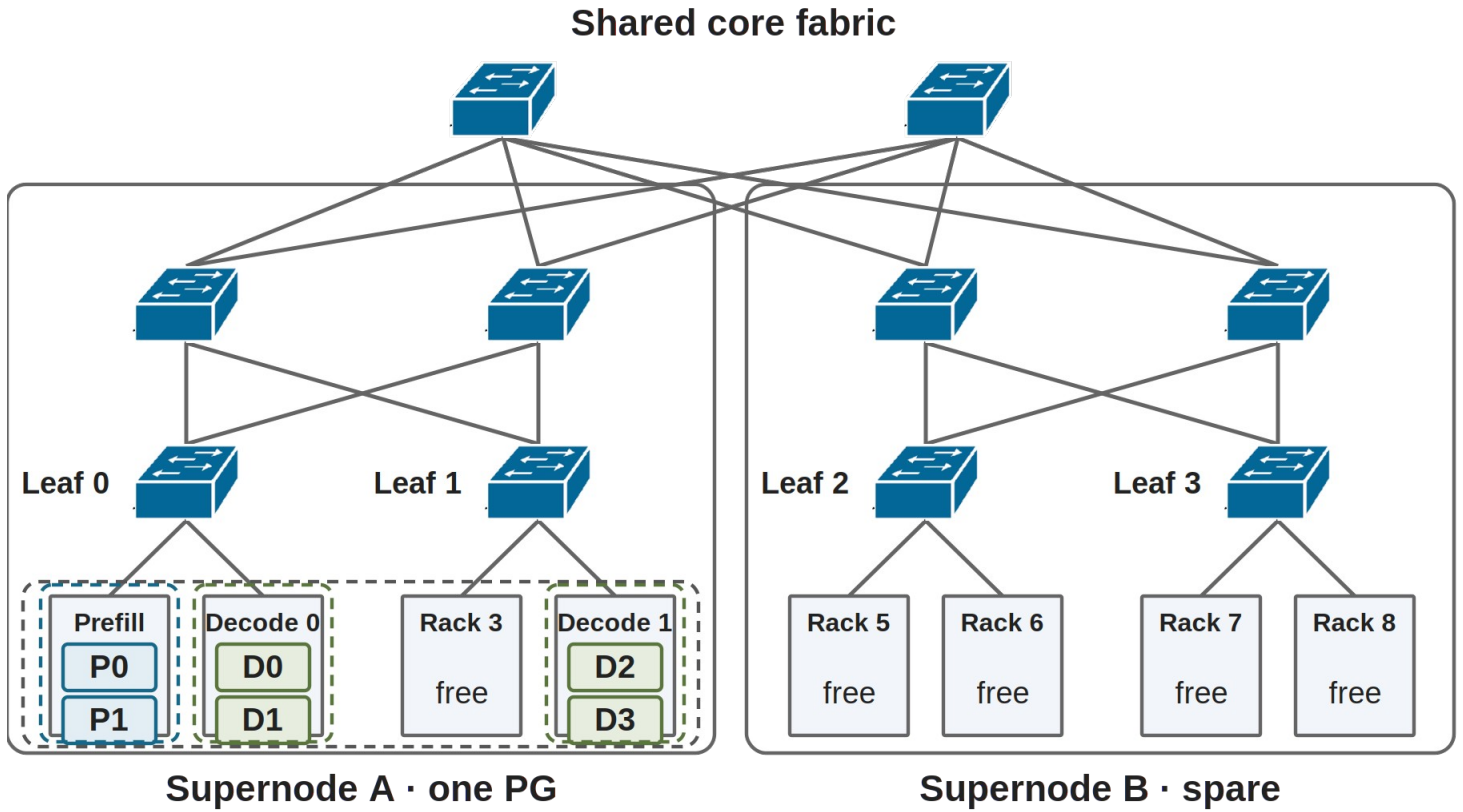
PodGroup.spec

```
networkTopology:
  mode: hard
  highestTierName: supernode
topologyAffinity:
  podGroupAntiAffinity:
    required:
    - podGroupSelector:
        matchLabels:
          app: model-a
      topologyTierName: supernode
```

Each PodGroup: app=model-a

Kthena integration proposed

SubGroup affinity and anti-affinity



Proposed

Affinity at supernode level

Prefill and Decode

Keep KV transfers local.

Anti-affinity at leaf level

Decode replicas

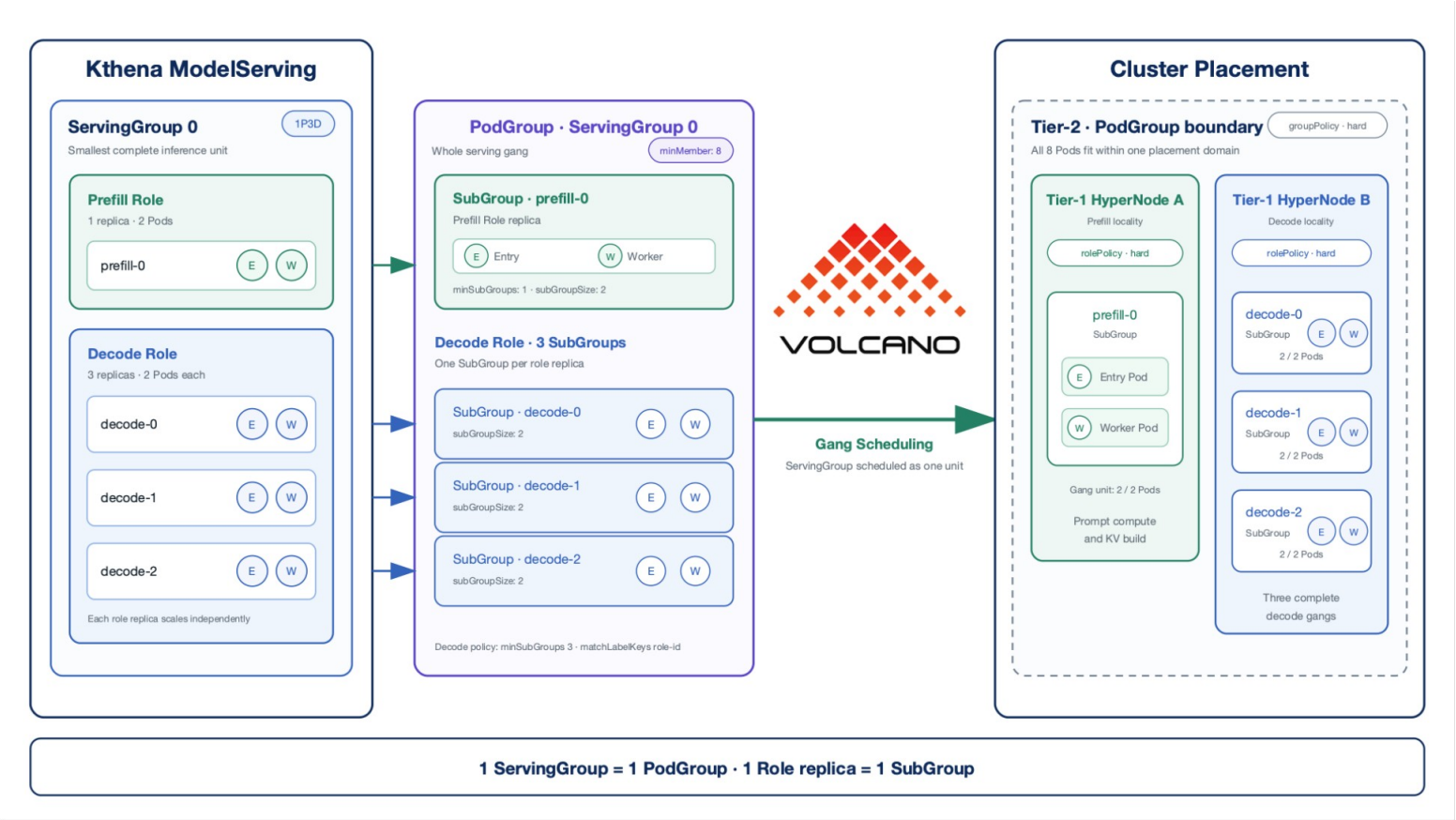
A leaf failure affects one Decode replica.

VOLCANO

03

Ecosystem

Kthena



ModelServing.spec

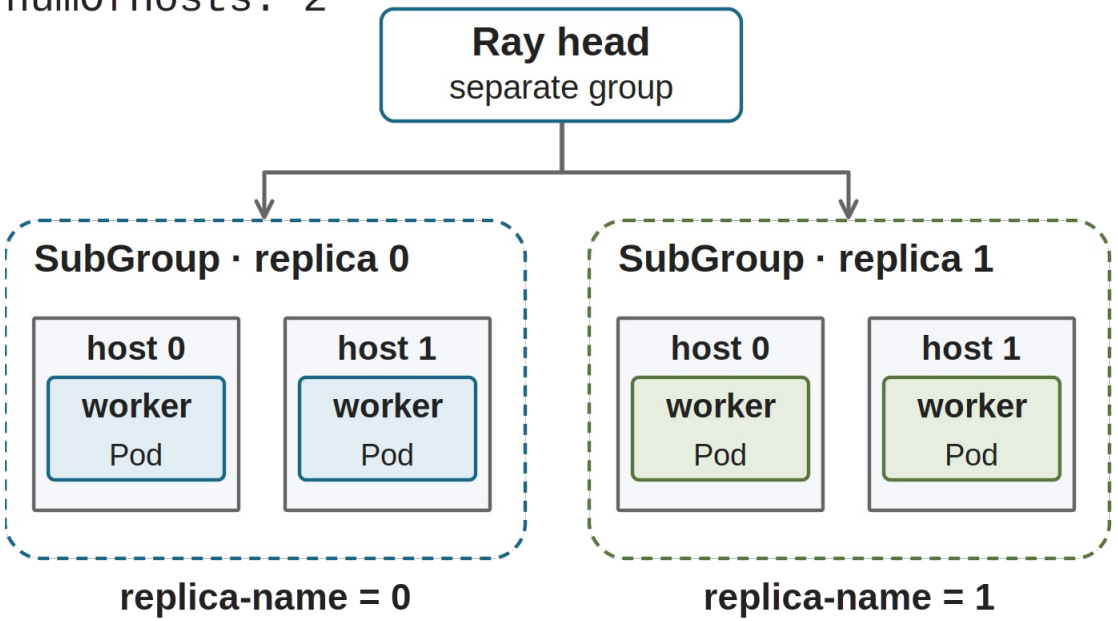
```
schedulerName: volcano
replicas: 1
template:
  gangPolicy:
    minRoleReplicas:
      prefill: 1
      decode: 3
  roles:
  - name: prefill
    replicas: 1
    workerReplicas: 1
  - name: decode
    replicas: 3
    workerReplicas: 1
```

Each role replica is one SubGroup.

KubeRay

Ray worker group

```
minReplicas: 2
numOfHosts: 2
```



PodGroup.spec

```
subGroupPolicy:
- name: worker
  labelSelector:
    matchLabels:
      ray.io/group: worker
  matchLabelKeys:
  - ray.io/worker-group-replica-name
  minSubGroups: 2
  subGroupSize: 2
```

One replica label identifies one SubGroup.
All hosts join the gang together.

Topology support in progress

Grove



Grove PodGang

PodGang.spec.podGroups

prefill `minReplicas: 2`

Pod 0 | Pod 1

decode `minReplicas: 4`

Pod 0 | Pod 1 | Pod 2 | Pod 3

one policy

one policy

Volcano PodGroup

PodGroup.spec.subGroupPolicy

prefill `subGroupSize: 2`

`minSubGroups: 1`

decode `subGroupSize: 4`

`minSubGroups: 1`

Each group becomes a complete SubGroup.

`minMember: 6`

Topology constraints not supported yet

Next integrations



RBG

In progress

RoleInstance → SubGroup

Keep each role instance runnable.



Kubeflow

Planned

Training groups and topology

Place distributed workers as complete groups.



LWS

Planned

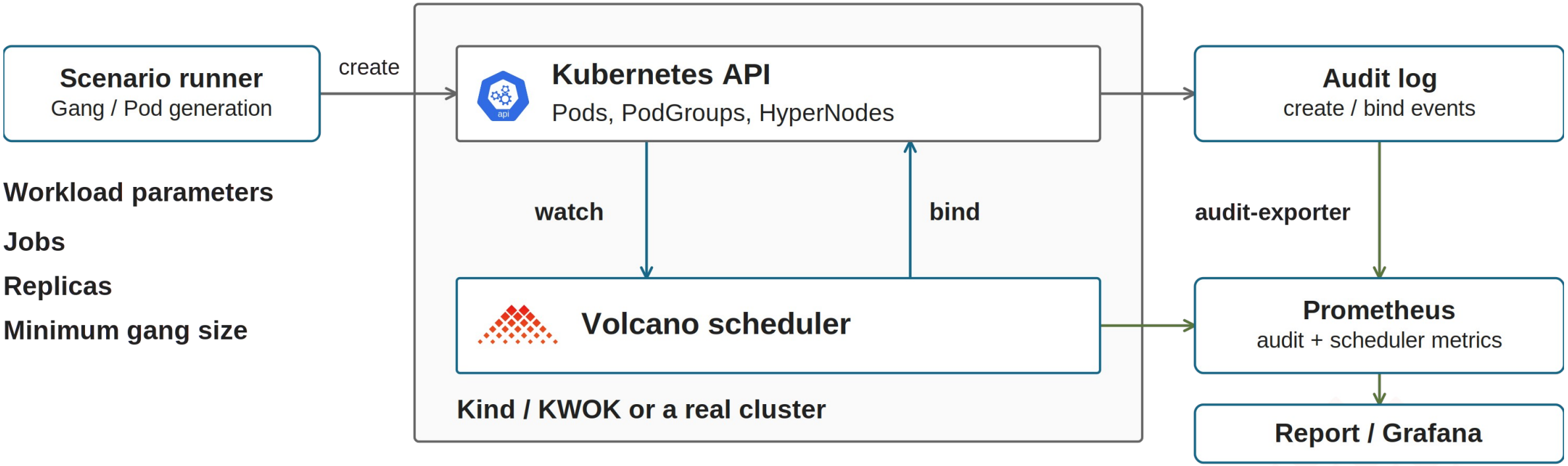
Multi-group support

Express relationships across role groups.

04

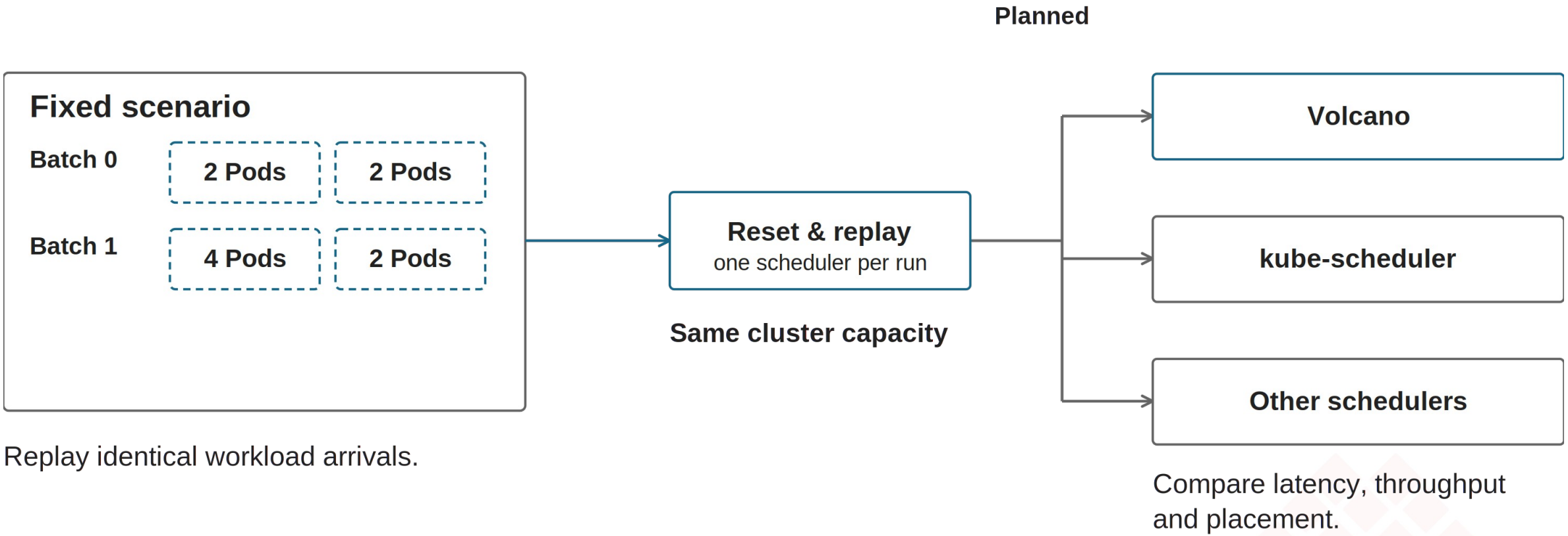
Benchmark

Benchmark framework



Use API events to measure scheduling latency and throughput.

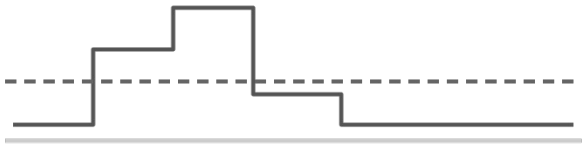
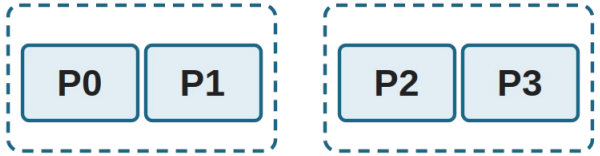
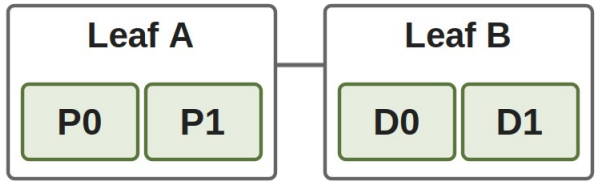
Comparing schedulers



Replay identical workload arrivals.

Compare latency, throughput and placement.

Benchmark scenarios

Scenario		Vary	Measure
Bursts		**Arrival rate** Burst above the available capacity.	**Tail scheduling latency**
Gang size		**Group size and fragmentation** Try small and large SubGroups.	**Complete gangs placed**
Topology		**Topology constraints** Vary tier limits and free domains.	**Placement correctness**

What’s next

- SubGroup affinity and anti-affinity
- RBG, Kubeflow and LWS integration
- Cross-scheduler and topology benchmarks

Volcano community



Thank you

volcano.sh

GitHub

Slack

Questions?